ChatGPT

KaamSetu PRD v1.0

KaamSetu v1.0 is a tightly scoped, closed-beta, invite-only, PWA-first product for one founder running a manually assisted hyperlocal service marketplace. The product must stay a single web application: Next.js App Router for the frontend and route handlers, Supabase for Auth, Postgres, Storage, and Row Level Security, and Vercel for deployment. Next.js supports the App Router, route handlers, and web-app manifest files; Supabase provides Auth, Postgres, Storage, and RLS; and Vercel's Hobby tier is a valid free starting point for a small beta. [1]

This PRD deliberately keeps customer operations anonymous and invite-gated, worker operations authenticated, and dispatch operations founder-controlled. Worker login uses a mobile-number UI over password-based Supabase Auth rather than SMS OTP, because Supabase's phone login requires an SMS provider, while password-based auth is built in. Manual WhatsApp Business messaging and optional manual Razorpay Payment Links remain external tools, not product integrations, because the WhatsApp Business app is free to download and intended for business-customer communication, and Razorpay Payment Links can be created from the dashboard or API without building a full checkout flow. [2]

Product overview

Product purpose

KaamSetu exists to do five things and only five things in MVP:

Capability	Definition
Onboard workers	Create invited worker accounts, collect basic KYC/profile data, approve or reject them
Accept customer requests	Let a customer submit a request through a simple closed-beta PWA flow
Match jobs	Let the founder review, price, and sequentially dispatch a request to one worker at a time
Track jobs	Maintain a durable job record from request submission to closure
Record payments and trust signals	Record whether payment happened, then collect rating or complaint

Product goals

Goal	Requirement
Low-friction customer booking	No mandatory customer account in MVP
Low-cost worker onboarding	No SMS OTP, no field ops dependency
Manual-first operations	Founder can run dispatch, approvals, payment confirmation, and complaint handling alone
High trust	Basic worker verification, job reference tracking, payment record, rating, complaint record, activity log
Low engineering complexity	One monolith, no microservices, no native apps, no real-time maps, no chat
Beta control	Invite-gated request access, serviceable localities only, serial dispatch only

Product non-goals

Out of scope for MVP	Reason
Native Android/iOS apps	PWA-first stack is locked
Customer sign-up / login / wallet	Adds friction and scope without launch necessity
Public worker browsing or choosing specific workers	Product is founder-matched, not marketplace-discovery-first
Auto-pricing engine	Founder controls pricing manually in triage
Parallel multi-worker bidding	Adds race conditions and operational complexity
In-app chat, calling infra, maps, live tracking	Not necessary to validate closed-beta workflow
Razorpay API integration, webhooks, escrow, subscriptions, ads, coupons, referrals	Explicitly excluded or delayed
SMS OTP, push notifications, WhatsApp API automation	Too costly or too complex for the solo-founder phase

User types

User type	Authentication	Access
Anonymous beta customer	No account; invite code + job ref + track code	Submit request, upload photos, track job, confirm payment, rate, complain

User type	Authentication	Access
Worker	Supabase Auth session	Login, complete KYC/profile, view offers, accept/decline, update job status, view earnings
Admin	Supabase Auth session with `role=admin`	Full founder control over approvals, requests, dispatch, jobs, payments, complaints, settings, logs

Core user journeys

Journey	End-to-end path
Customer booking	Landing Page → Invite Gate → New Request Form → Photo Upload → Request Submitted
Customer follow-up	Request Submitted → Track Job → Payment Confirmation → Rating & Complaint
Worker onboarding	Admin creates worker → Worker Login → KYC/Profile → Admin approval → Worker Dashboard
Job fulfillment	Request Queue → Triage → Dispatch Board → Worker accepts → Active Job → Status Update → Payment Ledger → Close
Issue handling	Track Job or Job Details → Complaint created → Complaints queue → Admin resolution

Critical product rule

At any time, one job can have only one active dispatch offer. Dispatch is serial, not parallel. This is a deliberate scope lock for v1.0.

Customer PWA

The customer PWA is an installable mobile web app with an invite-gated request flow and a token-lite tracking flow. PWA install metadata should be delivered through the Next.js manifest convention, but offline data sync is not required in MVP. [3]

Landing Page

Item	Spec
Purpose	Explain KaamSetu, establish trust, show supported services/localities, start request or track existing job
Entry conditions	Public root path `/`

Item	Spec
UI components	Hero banner, tagline, trust badges, supported services list, active localities list, “Start Request,” “Track Job,” and “WhatsApp Help” CTAs
Form fields	None
Validation rules	N/A
User actions	Proceed to invite gate; jump to tracking flow; open manual WhatsApp fallback
Success states	Invite gate opens; track flow opens; external WhatsApp link opens
Error states	Bootstrap data unavailable; service temporarily paused
API calls	`GET /api/public/bootstrap`

Invite Gate

Item	Spec
Purpose	Restrict booking to closed beta users and active localities
Entry conditions	Customer taps “Start Request”
UI components	Invite code input, continue button, simple helper text
Form fields	`invite_code`
Validation rules	Required; uppercase alphanumeric; length 6–12; must exist; must be active; not expired; usage count below limit; if locality-bound, must remain valid for chosen locality later
User actions	Validate code; continue
Success states	Invite code stored in session/local storage; proceed to New Request Form
Error states	Invalid code, expired code, exhausted code, beta paused
API calls	`POST /api/public/invite/validate`

New Request Form

Item	Spec
Purpose	Collect the minimum data needed to create a service request
Entry conditions	Valid invite code present in local session

Item	Spec
UI components	Full form, category picker, locality dropdown, address text area, description box, date/time selector, submit button
Form fields	`invite_code` hidden; `full_name`; `phone`; `alternate_phone` optional; `locality_id`; `address_text`; `landmark` optional; `service_category_id`; `description`; `preferred_date` optional; `preferred_time_slot`; `payment_preference` (`cash`, `upi`, `either`); conditional `workers_needed` and `shift_type` if category is daily-wage
Validation rules	`full_name` 2–80 chars; `phone` Indian 10-digit mobile; `alternate_phone` same pattern if present; `locality_id` must be serviceable; `address_text` 10–250 chars; `landmark` max 120 chars; `service_category_id` must be active; `description` 20–500 chars; `preferred_date` today or later, max 30 days ahead; `preferred_time_slot` enum; `payment_preference` enum; `workers_needed` integer 1–20 when required; `shift_type` enum (`half_day`, `full_day`) when required
User actions	Submit request; go back; edit fields
Success states	Job created with `booking_status=requested`; customer gets `job_ref` and one-time `track_code`; move to Photo Upload
Error states	Invalid invite; duplicate request within cooldown; locality not serviceable; rate limit exceeded
API calls	`GET /api/public/bootstrap`; `POST /api/public/requests`

Photo Upload

Item	Spec
Purpose	Attach optional issue photos to improve triage quality
Entry conditions	Request already created; `job_ref` and `public_id` returned
UI components	Photo picker, preview grid, remove action, skip button, continue button
Form fields	Up to 3 files; no caption field in MVP
Validation rules	Optional; max 3 files; `image/jpeg`, `image/png`, `image/webp`; max 5 MB each; reject HEIC unless browser converts client-side; no PDF
User actions	Upload; remove; skip; continue
Success states	All files stored and linked to `job_media`; move to Request Submitted
Error states	Unsupported file type; too many files; upload timeout; partial upload failure

Item	Spec
API calls	`POST /api/public/jobs/{publicId}/media`

Request Submitted

Item	Spec
Purpose	Confirm creation and hand the customer the information needed to track the job later
Entry conditions	Request creation succeeded
UI components	Success message, `job_ref`, `track_code`, request summary, “Track Job,” “Copy Details,” and “WhatsApp Help” buttons
Form fields	None
Validation rules	N/A
User actions	Copy details; open tracking flow; open WhatsApp help
Success states	Customer sees confirmation and next steps
Error states	Lost session on refresh; if refreshed, customer must use Track Job lookup with `job_ref + phone + track_code`
API calls	Uses `POST /api/public/requests` response; optional `POST /api/public/jobs/lookup` on refresh recovery

Track Job

Item	Spec
Purpose	Let a customer retrieve current status without creating an account
Entry conditions	Customer has `job_ref`, phone, and `track_code`
UI components	Lookup form, status timeline, assigned worker summary, amount summary, action buttons for payment/rating/complaint depending on status
Form fields	Lookup state: `job_ref`, `phone`, `track_code`
Validation rules	`job_ref` required, format `KS-######`; `phone` 10-digit mobile; `track_code` 6 digits; 5 failed attempts per IP per hour max
User actions	Lookup job; refresh; go to payment confirmation if payment due; go to rating/complaint if eligible

Item	Spec
Success states	Status timeline shown; limited worker details visible only after assignment
Error states	Not found; wrong credentials; too many failed attempts; cancelled job
API calls	`POST /api/public/jobs/lookup`

Payment Confirmation

Item	Spec
Purpose	Record whether the customer paid, how they paid, and any reference number
Entry conditions	Job completed; final amount available; payment not yet admin-confirmed
UI components	Amount due card, payment method selector, UPI ref field, note field, optional Razorpay link card if admin attached one, submit button
Form fields	`job_ref`; `phone`; `track_code`; `payment_method_used` (`cash`, `upi`, `razorpay_link`, `bank_transfer`); `amount_paid`; `external_reference` optional but required for non-cash except `razorpay_link`; `note` optional; `confirm_checkbox`
Validation rules	Credentials required; `amount_paid` must equal `final_amount`; method enum required; `external_reference` 4–80 chars for non-cash when applicable; one confirmation submission per state change
User actions	Submit payment confirmation; open external payment link if present
Success states	`payments.status=customer_marked_paid`; thank-you state shown
Error states	Amount mismatch; payment not due yet; job already payment-confirmed; invalid credentials
API calls	`POST /api/public/jobs/{publicId}/payments/confirm`

Rating & Complaint

Item	Spec
Purpose	Collect trust signals after job completion and provide a simple issue-reporting path
Entry conditions	Job `booking_status=closed` for rating; complaint allowed when `assigned`, `in_progress`, `completed`, `closed`, or `disputed`
UI components	Ratings card, text review box, complaint type chips, complaint description box, separate submit actions

Item	Spec
Form fields	Rating: `job_ref`, `phone`, `track_code`, `overall_rating`, `punctuality_rating` optional, `behavior_rating` optional, `quality_rating` optional, `review_text` optional. Complaint: `job_ref`, `phone`, `track_code`, `complaint_type`, `description`
Validation rules	Credentials required; overall rating 1–5; sub-ratings 1–5 if present; review max 300 chars; complaint type enum; complaint description 20–500 chars; one rating per job; one complaint record per job in MVP
User actions	Submit rating; submit complaint
Success states	Rating saved; complaint reference shown
Error states	Duplicate rating; duplicate complaint; job not eligible; invalid credentials
API calls	`POST /api/public/jobs/{publicId}/ratings`; `POST /api/public/jobs/{publicId}/complaints`

Worker PWA

The worker PWA is authenticated and intentionally simple. The login screen shows mobile number + password, but the backend uses Supabase password auth under the hood. This avoids SMS cost and complexity in MVP, because Supabase phone login requires an SMS provider while password auth is available directly. [4]

Login

Item	Spec
Purpose	Authenticate invited workers
Fields	`mobile_number`, `password`
Validation	Mobile must be 10 digits; password 8–32 chars; account must exist and not be suspended
Actions	Login
States	Idle; loading; invalid credentials; under-review account; suspended account; success
API calls	`POST /api/worker/auth/login`

KYC/Profile

Item	Spec
Purpose	Collect minimum worker profile and KYC data before activation

Item	Spec
Fields	`full_name`; `mobile_number` read-only; `whatsapp_number`; `primary_category_id`; `secondary_category_ids` max 2; `locality_id`; `address_text`; `years_experience`; `supported_job_modes` (`fixed`, `quote`, `daily_wage`); `has_own_tools`; `government_id_type`; `government_id_last4`; `consent_checkbox`; document uploads: `profile_photo`, `government_id_front`, `government_id_back` optional
Validation	Name 2–80 chars; WhatsApp number 10 digits; primary category required; secondary max 2 distinct active categories; locality required; address 10–250 chars; years experience 0–40; job modes at least 1; ID type enum; ID last 4 digits only; consent required; documents required per approval rule
Actions	Save profile; upload documents; submit for review
States	Incomplete; saved draft; under review; rejected with reason; approved
API calls	`GET /api/worker/me`; `PUT /api/worker/profile`; `POST /api/worker/documents`

Dashboard

Item	Spec
Purpose	Give worker a single view of current status and available work
Fields	None editable in MVP
Validation	N/A
Actions	Open pending offer; open active job; open earnings log; retry refresh
States	Under review; approved no offers; pending offer; active job; no network
API calls	`GET /api/worker/dashboard`; `GET /api/worker/offers`

Offer Details

Item	Spec
Purpose	Let worker inspect one dispatch offer and accept or decline it
Fields	None editable except optional decline note
Validation	Offer must belong to logged-in worker; offer must be `sent`; offer must not be expired/cancelled
Actions	Accept; decline
States	Pending; accepted; declined; expired; withdrawn; job already assigned

Item	Spec
API calls	`GET /api/worker/offers/{dispatchId}`; `POST /api/worker/offers/{dispatchId}/accept`; `POST /api/worker/offers/{dispatchId}/decline`

Active Job

Item	Spec
Purpose	Show live assigned job details needed to execute work
Fields	None directly editable
Validation	Worker must be assigned to the job
Actions	Call customer via phone link; open status update; refresh
States	Assigned; in progress; completed awaiting admin close; disputed
API calls	`GET /api/worker/jobs/{jobId}`

Status Update

Item	Spec
Purpose	Let worker move the job through operational milestones
Fields	`status_action` (`start_work`, `complete_work`, `report_issue`); `note` optional; `amount_collected` optional when completing; `payment_method` optional when completing; `external_reference` optional for UPI/bank; `completion_photo` optional
Validation	Only assigned worker can update; `start_work` only from `assigned`; `complete_work` only from `in_progress`; `report_issue` allowed from `assigned` or `in_progress`; if `complete_work`, `amount_collected` numeric >= 0; if payment method is non-cash and payment recorded, external reference required
Actions	Submit status; upload optional completion photo
States	Start success; completion success; issue reported; invalid transition; upload failure
API calls	`POST /api/worker/jobs/{jobId}/media`; `POST /api/worker/jobs/{jobId}/status`

Earnings Log

Item	Spec
Purpose	Show jobs done and what the worker has earned according to KaamSetu records
Fields	Filter only: `from_date`, `to_date` optional

Item	Spec
Validation	Date range max 90 days in MVP
Actions	Refresh
States	Empty; with data; network error
API calls	`GET /api/worker/earnings`

Admin Panel

The admin panel is founder-operated and manual-first. It exists to run the marketplace without hiring an operations team. All business APIs should run within the same Next.js codebase through route handlers, while media should remain in private Supabase Storage buckets protected by strict access rules. Supabase Storage supports private buckets and bucket-level upload restrictions, and RLS is the core authorization primitive for exposed data. [5]

Dashboard

Item	Spec
Purpose	Single founder control center
Permissions	Admin only
Filters	Date window, locality
Tables / cards	New requests, jobs needing dispatch, active jobs, payments awaiting confirmation, open complaints, workers pending approval
Actions	Open request queue, dispatch board, complaints, payments
API calls	`GET /api/admin/dashboard`

Worker Approvals

Item	Spec
Purpose	Review submitted worker profiles and KYC docs; create provisional worker records
Permissions	Admin only
Filters	`approval_status`, locality, category
Tables	Pending workers table with name, phone, category, locality, profile completeness, doc status
Actions	View worker detail; approve; reject with reason; create provisional worker; reset password

Item	Spec
API calls	`GET /api/admin/workers?approval_status=under_review`; `POST /api/admin/workers`; `GET /api/admin/workers/{workerId}`; `POST /api/admin/workers/{workerId}/approve`; `POST /api/admin/workers/{workerId}/reject`; `POST /api/admin/workers/{workerId}/reset-password`

Worker Directory

Item	Spec
Purpose	View all workers and operationally manage the supply side
Permissions	Admin only
Filters	Category, locality, approval status, phone search
Tables	Worker list with code, name, primary skill, locality, status, jobs completed, avg rating
Actions	View detail; deactivate/suspend through profile status patch; reset password
API calls	`GET /api/admin/workers`; `GET /api/admin/workers/{workerId}`; `POST /api/admin/workers/{workerId}/reset-password`

Request Queue

Item	Spec
Purpose	Review all new or manually created requests before dispatch
Permissions	Admin only
Filters	`request_source`, category, locality, booking status
Tables	Request list with job ref, customer, category, locality, created time, requested slot
Actions	Create assisted request; open details; triage; cancel
API calls	`GET /api/admin/jobs?queue=requests`; `POST /api/admin/jobs`; `GET /api/admin/jobs/{jobId}`; `PATCH /api/admin/jobs/{jobId}/triage`; `PATCH /api/admin/jobs/{jobId}/status`

Dispatch Board

Item	Spec
Purpose	Sequentially send one offer at a time and monitor pending responses
Permissions	Admin only
Filters	Category, locality, dispatch status

Item	Spec
Tables	Jobs needing worker; current offer; offer expiry; last attempt; suggested workers list in side panel
Actions	Create dispatch attempt; mark offer expired/withdrawn; manually assign worker; mark no worker found
API calls	`GET /api/admin/jobs?queue=dispatch`; `POST /api/admin/jobs/{jobId}/dispatch-attempts`; `PATCH /api/admin/dispatch-attempts/{dispatchId}`; `POST /api/admin/jobs/{jobId}/assign`; `PATCH /api/admin/jobs/{jobId}/status`

Job Details

Item	Spec
Purpose	Full operational record for one job
Permissions	Admin only
Filters	N/A
Tables	Job summary, customer card, worker card, media gallery, dispatch history, payment block, activity feed
Actions	Edit triage fields; assign/unassign before start; change status; paste payment link; confirm payment; open complaint if needed
API calls	`GET /api/admin/jobs/{jobId}`; `PATCH /api/admin/jobs/{jobId}/triage`; `POST /api/admin/jobs/{jobId}/assign`; `PATCH /api/admin/jobs/{jobId}/status`; `POST /api/admin/jobs/{jobId}/payment-link`

Payment Ledger

Item	Spec
Purpose	Track payment state for every completed job
Permissions	Admin only
Filters	Payment status, method, date range, locality
Tables	Payment list with job ref, customer, worker, amount, method, status, reference
Actions	Confirm payment; mark failed; waive; add/edit reference and notes
API calls	`GET /api/admin/payments`; `PATCH /api/admin/payments/{paymentId}`

Complaints

Item	Spec
Purpose	Review and resolve customer issues
Permissions	Admin only
Filters	`status`, complaint type, date range
Tables	Complaint list with complaint ref, job ref, customer, type, status, created time
Actions	Open detail; move to under review; resolve; dismiss; add resolution note
API calls	`GET /api/admin/complaints`; `GET /api/admin/complaints/{complaintId}`; `PATCH /api/admin/complaints/{complaintId}`

Settings

Item	Spec
Purpose	Control locality availability, category availability, and invite code management
Permissions	Admin only
Filters	Category/locality active state; invite code type
Tables	Active categories; serviceable localities; invite code list with usage
Actions	Activate/deactivate category; activate/deactivate locality; create invite code; disable invite code
API calls	`GET /api/admin/settings`; `PATCH /api/admin/settings/categories/{categoryId}`; `PATCH /api/admin/settings/localities/{localityId}`; `POST /api/admin/invite-codes`; `PATCH /api/admin/invite-codes/{inviteId}`

Activity Logs

Item	Spec
Purpose	Audit trail for operational and security events
Permissions	Admin only
Filters	Actor type, action, entity type, date range
Tables	Activity log list with actor, action, entity, timestamp, metadata preview
Actions	View log detail only
API calls	`GET /api/admin/activity-logs`

Data, APIs, and state machines

All business data should live in Supabase Postgres, all uploads in private Supabase Storage buckets, and all mutations should go through Next.js route handlers for centralized validation, logging, and secret management. Supabase explicitly recommends RLS for exposed schemas, and Storage integrates access control through the same policy model. [6]

Data model

Table	Purpose	Columns and data types	Required fields
`auth_users`	Supabase-managed auth identity store; do not recreate in `public`	`id uuid PK`; `email text`; `phone text`; `raw_app_meta_data jsonb`; `raw_user_meta_data jsonb`; `created_at timestamptz`; `last_sign_in_at timestamptz`	`id`; for workers/admin, `email` required in practice
`customer_profiles`	Stores request-side customer identity without requiring customer login	`id uuid PK`; `auth_user_id uuid null`; `full_name text`; `phone text`; `alternate_phone text null`; `language_code text`; `locality_id uuid`; `default_address_text text null`; `landmark text null`; `invite_code_id uuid null`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `full_name`, `phone`, `locality_id`, `language_code`

Table	Purpose	Columns and data types	Required fields
worker_profiles	Core worker record used for matching and trust	id uuid PK; auth_user_id uuid unique; worker_code text unique; full_name text; phone text unique; whatsapp_number text; locality_id uuid; address_text text; primary_category_id uuid; secondary_category_ids uuid[] null; supported_job_modes text[]; years_experience integer; has_own_tools boolean; government_id_type text; government_id_last4 text; approval_status text; rejection_reason text null; approved_at timestamptz null; created_at timestamptz; updated_at timestamptz	id, auth_user_id, worker_code, full_name, phone, locality_id, primary_category_id, approval_status
worker_documents	Stores file metadata for worker KYC docs kept in private Storage	id uuid PK; worker_profile_id uuid; document_type text; storage_path text; mime_type text; file_size_bytes integer; verification_status text; verified_by uuid null; verified_at timestamptz null; rejection_reason text null; created_at timestamptz	id, worker_profile_id, document_type, storage_path, mime_type, verification_status
service_categories	Seeded list of service types supported in beta	id uuid PK; slug text unique; name_en text; name_hi text; pricing_type_default text; requires_shift_fields boolean; is_active boolean; sort_order integer; created_at timestamptz; updated_at timestamptz	id, slug, name_en, pricing_type_default, is_active

Table	Purpose	Columns and data types	Required fields
`localities`	Serviceable area list for a single city launch or adjacent areas	`id uuid PK`; `city text`; `state text`; `name text`; `pincode text null`; `is_serviceable boolean`; `is_active boolean`; `sort_order integer`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `city`, `name`, `is_serviceable`, `is_active`

Table	Purpose	Columns and data types	Required fields
`jobs`	Master operational table for every request	`id uuid PK`; `public_id text unique`; `tracking_code_hash text`; `customer_profile_id uuid`; `invite_code_id uuid null`; `service_category_id uuid`; `locality_id uuid`; `request_source text`; `pricing_type text`; `description text`; `address_text text`; `landmark text null`; `preferred_date date null`; `preferred_time_slot text`; `workers_needed integer null`; `shift_type text null`; `booking_status text`; `dispatch_status text`; `payment_status text`; `complaint_status text`; `assigned_worker_id uuid null`; `assigned_dispatch_attempt_id uuid null`; `estimated_amount numeric(10,2) null`; `final_amount numeric(10,2) null`; `worker_payable_amount numeric(10,2) null`; `customer_payment_preference text`; `admin_notes text null`; `status_reason text null`; `requested_at timestamptz`; `assigned_at timestamptz null`; `started_at timestamptz null`; `completed_at timestamptz null`; `closed_at timestamptz null`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `public_id`, `tracking_code_hash`, `customer_profile_id`, `service_category_id`, `locality_id`, `request_source`, `pricing_type`, `description`, `address_text`, `preferred_time_slot`, `booking_status`, `dispatch_status`, `payment_status`, `complaint_status`, `customer_payment_preference`, `requested_at`

Table	Purpose	Columns and data types	Required fields
`job_media`	File metadata for issue photos and completion evidence	`id uuid PK`; `job_id uuid`; `uploaded_by_role text`; `uploaded_by_user_id uuid null`; `media_kind text`; `storage_path text`; `mime_type text`; `file_size_bytes integer`; `created_at timestamptz`	`id`, `job_id`, `uploaded_by_role`, `media_kind`, `storage_path`, `mime_type`
`dispatch_attempts`	History of every offer attempt sent to a worker	`id uuid PK`; `job_id uuid`; `worker_profile_id uuid`; `offer_status text`; `contact_method text`; `offered_amount numeric(10,2) null`; `offer_expires_at timestamptz null`; `response_note text null`; `sent_at timestamptz`; `responded_at timestamptz null`; `created_by_admin_id uuid`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `job_id`, `worker_profile_id`, `offer_status`, `contact_method`, `sent_at`, `created_by_admin_id`
`payments`	Single current payment record per job in MVP	`id uuid PK`; `job_id uuid unique`; `amount numeric(10,2)`; `payment_method text null`; `status text`; `link_url text null`; `external_reference text null`; `reported_by_role text null`; `reported_by_user_id uuid null`; `reported_at timestamptz null`; `confirmed_by uuid null`; `confirmed_at timestamptz null`; `note text null`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `job_id`, `amount`, `status`

Table	Purpose	Columns and data types	Required fields
`ratings`	Single customer-to-worker rating after closure	`id uuid PK`; `job_id uuid unique`; `customer_profile_id uuid`; `worker_profile_id uuid`; `overall_rating smallint`; `punctuality_rating smallint null`; `behavior_rating smallint null`; `quality_rating smallint null`; `review_text text null`; `created_at timestamptz`	`id`, `job_id`, `customer_profile_id`, `worker_profile_id`, `overall_rating`
`complaints`	Single complaint record per job in MVP	`id uuid PK`; `complaint_ref text unique`; `job_id uuid unique`; `customer_profile_id uuid null`; `worker_profile_id uuid null`; `complaint_type text`; `description text`; `status text`; `resolution_note text null`; `resolved_by uuid null`; `resolved_at timestamptz null`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `complaint_ref`, `job_id`, `complaint_type`, `description`, `status`
`invite_codes`	Controls closed-beta access	`id uuid PK`; `code text unique`; `code_type text`; `locality_id uuid null`; `max_uses integer`; `used_count integer`; `expires_at timestamptz null`; `is_active boolean`; `created_by uuid null`; `created_at timestamptz`; `updated_at timestamptz`	`id`, `code`, `code_type`, `max_uses`, `used_count`, `is_active`
`activity_logs`	Immutable audit trail	`id uuid PK`; `actor_type text`; `actor_user_id uuid null`; `actor_label text null`; `entity_type text`; `entity_id uuid null`; `action text`; `metadata jsonb`; `ip_hash text null`; `user_agent text null`; `created_at timestamptz`	`id`, `actor_type`, `entity_type`, `action`, `created_at`

API standards

All success responses must use:

```
{
  "success": true,
  "data": {}
}
```

All error responses must use:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human readable message",
    "field_errors": {
      "field_name": "Specific issue"
    }
  }
}
```

Standard error codes allowed in MVP: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`, `RATE_LIMITED`, `UPLOAD_ERROR`, `INVALID_STATE_TRANSITION`, `INTERNAL_ERROR`.

Public endpoints

Method	URL	Request body
GET	`/api/public/bootstrap`	Query: none
POST	`/api/public/invite/validate`	`invite_code`

Method	URL	Request body
POST	`/api/public/requests`	`invite_code`, `customer:{full_name,phone,alternate_phone?,locality_id,addres` `{service_category_id,description,preferred_date?,preferred_time_slot,paymer`
POST	`/api/public/jobs/{publicId}/media`	`multipart/form-data`: `files[]`, `job_ref`, `phone`, `track_code`
POST	`/api/public/jobs/lookup`	`job_ref`, `phone`, `track_code`
POST	`/api/public/jobs/{publicId}/payments/confirm`	`job_ref`, `phone`, `track_code`, `payment_method_used`, `amount_paid`, `external_re`
POST	`/api/public/jobs/{publicId}/ratings`	`job_ref`, `phone`, `track_code`, `overall_rating`, `punctuality_rating?`, `behavior`
POST	`/api/public/jobs/{publicId}/complaints`	`job_ref`, `phone`, `track_code`, `complaint_type`, `description`

Worker endpoints

Method	URL	Request body	Response body	Validation rules
POST	`/api/worker/auth/login`	`mobile_number`, `password`	`session_created`, `worker_profile_status`, `redirect_to`	Normalize mobile; map t deterministic auth email password required; login suspended
GET	`/api/worker/me`	Query: none	`worker_profile`, `document_summary`, `approval_status`	Authenticated worker or
PUT	`/api/worker/profile`	`full_name`, `whatsapp_number`, `primary_category_id`, `secondary_category_ids?`, `locality_id`, `address_text`, `years_experience`, `supported_job_modes`, `has_own_tools`, `government_id_type`, `government_id_last4`, `consent_checkbox`	`worker_profile_id`, `approval_status`, `updated_at`	Profile editable until app secondary categories ma categories only
POST	`/api/worker/documents`	`multipart/form-data`: `document_type`, `file`	`document_id`, `verification_status`	Allowed document types private bucket only; max PNG/PDF
GET	`/api/worker/dashboard`	Query: none	`approval_status`, `pending_offers_count`, `active_job_summary|null`, `recent_earnings_summary`	Authenticated worker or
GET	`/api/worker/offers`	Query: optional `status=sent`	`offers[]`	Only offers for logged-in
GET	`/api/worker/offers/{dispatchId}`	Query: none	`offer_detail`, `job_summary`, `expires_at`	Offer must belong to wo
POST	`/api/worker/offers/{dispatchId}/accept`	`note?`	`accepted=true`, `job_id`, `booking_status`, `dispatch_status`	Offer status must be `se` expired; job unassigned

Method	URL	Request body	Response body	Validation rules
POST	`/api/worker/offers/{dispatchId}/decline`	`note?`	`declined=true`, `dispatch_status`	Offer status must be `se` max 200 chars if present
GET	`/api/worker/jobs/{jobId}`	Query: none	`job_detail`, `customer_contact`, `payment_snapshot`, `media[]`	Only assigned worker ca
POST	`/api/worker/jobs/{jobId}/media`	`multipart/form-data`: `media_kind`, `file`	`media_id`, `storage_path`	Only assigned worker; `media_kind=completi` image types only; max 5
POST	`/api/worker/jobs/{jobId}/status`	`status_action`, `note?`, `amount_collected?`, `payment_method?`, `external_reference?`, `completion_media_ids?`	`job_id`, `booking_status`, `payment_status`, `updated_at`	Allowed transitions only; requires current booking `in_progress`; paymer validated if provided
GET	`/api/worker/earnings`	Query: `from_date?`, `to_date?`	`earnings[]`, `summary`	Max 90-day range; work own jobs

Admin endpoints

Method	URL	Request body	Response body	Validation rules
GET	`/api/admin/dashboard`	Query: `from_date?`, `to_date?`, `locality_id?`	KPI cards + top queues	Admin only
GET	`/api/admin/workers`	Query: `approval_status?`, `locality_id?`, `category_id?`, `search?`, `page?`, `page_size?`	`workers[]`, `pagination`	Admin only
POST	`/api/admin/workers`	`phone`, `temporary_password`, `full_name?`, `primary_category_id?`, `locality_id?`	`worker_profile_id`, `worker_code`, `approval_status=invited`	Phone unique; pa 32 chars; create a provisional profil
GET	`/api/admin/workers/{workerId}`	Query: none	Full worker profile + docs + stats	Admin only

Method	URL	Request body	Response body	Validation rules
POST	`/api/admin/workers/{workerId}/approve`	`note?`	`approval_status=approved`, `approved_at`	Required docs mu
POST	`/api/admin/workers/{workerId}/reject`	`reason`	`approval_status=rejected`	Reason 5–300 cha
POST	`/api/admin/workers/{workerId}/reset-password`	`new_temporary_password`	`reset=true`	Password 8–32 ch
GET	`/api/admin/jobs`	Query: `queue?`, `booking_status?`, `dispatch_status?`, `payment_status?`, `locality_id?`, `category_id?`, `search?`, `page?`, `page_size?`	`jobs[]`, `pagination`	Admin only
POST	`/api/admin/jobs`	`source`, `customer`, `job`, `invite_code?`	`job_id`, `job_ref`, `track_code`, `booking_status=requested`	Same validation a request create; so enum `whatsapp_assi` `call_assisted` `admin_manual`
GET	`/api/admin/jobs/{jobId}`	Query: none	Full job details + media + dispatch history + payment + logs summary	Admin only
PATCH	`/api/admin/jobs/{jobId}/triage`	`service_category_id?`, `pricing_type?`, `estimated_amount?`, `final_amount?`, `workers_needed?`, `shift_type?`, `admin_notes?`, `booking_status?`	Updated job snapshot	Can only triage b `in_progress`; type enum; amou numeric >= 0

Method	URL	Request body	Response body	Validation rules
POST	`/api/admin/jobs/{jobId}/dispatch-attempts`	`worker_profile_id`, `contact_method`, `offered_amount?`, `offer_expires_at?`, `note?`	`dispatch_id`, `offer_status=sent`, `dispatch_status=offer_pending`	Job must be triag dispatchable; onl active sent offer/ worker must be a and match localit manually
PATCH	`/api/admin/dispatch-attempts/{dispatchId}`	`offer_status`, `response_note?`	Updated dispatch attempt	Allowed statuses: `expired`, `with` `accepted_manu` `declined_manu`
POST	`/api/admin/jobs/{jobId}/assign`	`worker_profile_id`, `dispatch_id?`, `note?`	`assigned=true`, `booking_status=assigned`	Only before start approved; if dispa supplied, it must same job/worker
PATCH	`/api/admin/jobs/{jobId}/status`	`booking_status`, `status_reason?`	Updated job snapshot	Allowed transitio cancellations req reason; `closed` payment confirm waived
GET	`/api/admin/payments`	Query: `status?`, `method?`, `from_date?`, `to_date?`, `locality_id?`, `page?`, `page_size?`	`payments[]`, `pagination`	Admin only
PATCH	`/api/admin/payments/{paymentId}`	`status`, `payment_method?`, `external_reference?`, `note?`	Updated payment snapshot	Allowed states: `payment_link_` `customer_mark` `admin_confirm` `failed`, `waive`
POST	`/api/admin/jobs/{jobId}/payment-link`	`link_url`, `amount`, `note?`	`payment_id`, `status=payment_link_sent`	Manual link only; amount > 0
GET	`/api/admin/complaints`	Query: `status?`, `complaint_type?`, `from_date?`, `to_date?`, `page?`, `page_size?`	`complaints[]`, `pagination`	Admin only
GET	`/api/admin/complaints/{complaintId}`	Query: none	Full complaint + job snapshot + resolution history	Admin only

Method	URL	Request body	Response body	Validation rules
PATCH	`/api/admin/complaints/{complaintId}`	`status`, `resolution_note?`	Updated complaint snapshot	Allowed transitio resolution note re `resolved` or `dismissed`
GET	`/api/admin/settings`	Query: none	`service_categories[]`, `localities[]`, `invite_codes[]`	Admin only
PATCH	`/api/admin/settings/categories/{categoryId}`	`is_active`, `name_en?`, `name_hi?`, `pricing_type_default?`, `requires_shift_fields?`	Updated category	Category must re unique by slug; a cannot lose refer category row
PATCH	`/api/admin/settings/localities/{localityId}`	`is_active`, `is_serviceable`, `name?`	Updated locality	Existing jobs rem even if locality lat deactivated
POST	`/api/admin/invite-codes`	`code_type`, `locality_id?`, `max_uses`, `expires_at?`, `is_active`	`invite_code_id`, `code`	Code auto-gener server-side; max integer > 0
PATCH	`/api/admin/invite-codes/{inviteId}`	`is_active`, `expires_at?`, `max_uses?`	Updated invite code	Used count cann new max uses
GET	`/api/admin/activity-logs`	Query: `actor_type?`, `entity_type?`, `action?`, `from_date?`, `to_date?`, `page?`, `page_size?`	`logs[]`, `pagination`	Admin only

State machines

The system must keep four separate state surfaces: `booking_status`, `dispatch_status`, `payment_status`, and complaint `status`. UI badges should derive from these fields rather than from free-text notes.

Booking state machine

State	Meaning	Allowed transitions	Invalid transitions
`requested`	Request created but not yet reviewed	`validated`, `cancelled`	`assigned`, `in_progress`, `completed`, `closed`

State	Meaning	Allowed transitions	Invalid transitions
`validated`	Admin reviewed and request is dispatch-ready	`dispatching`, `cancelled`	`completed`, `closed`, direct `in_progress`
`dispatching`	Founder is seeking worker	`assigned`, `cancelled`	`completed`, `closed`
`assigned`	Worker assigned but work not started	`in_progress`, `cancelled`, `disputed`	`completed`, `closed`
`in_progress`	Worker started job	`completed`, `disputed`	`validated`, `dispatching`, `cancelled`
`completed`	Work completed; payment may still be pending	`closed`, `disputed`	`assigned`, `in_progress`, `cancelled`
`disputed`	Issue requires admin resolution	`completed`, `closed`, `cancelled`	`validated`, `dispatching`
`closed`	Final terminal success state	none	Any transition
`cancelled`	Terminal non-completion state	none	Any transition

Dispatch state machine

State	Meaning	Allowed transitions	Invalid transitions
`not_started`	No worker outreach yet	`offer_pending`, `failed`, `stopped`	`assigned`
`offer_pending`	Exactly one active offer is open	`assigned`, `not_started`, `failed`, `stopped`	another simultaneous `offer_pending` to different worker
`assigned`	Offer accepted and worker attached	`stopped` only if admin explicitly removes before start	`offer_pending` without first stopping current assignment
`failed`	No suitable worker found	`offer_pending`, `stopped`	`assigned` without fresh offer/manual assign

State	Meaning	Allowed transitions	Invalid transitions
`stopped`	Dispatch intentionally halted because booking cancelled or manually paused	`offer_pending` only if booking reopened appropriately	`assigned` directly if booking cancelled

Payment state machine

State	Meaning	Allowed transitions	Invalid transitions
`not_due`	Job not yet payable	`due`	Any paid state
`due`	Job completed and amount due	`payment_link_sent`, `customer_marked_paid`, `waived`	`admin_confirmed_paid` directly without report/ confirmation
`payment_link_sent`	Manual Razorpay link stored and shared	`customer_marked_paid`, `failed`, `waived`	`admin_confirmed_paid` without evidence or manual confirmation
`customer_marked_paid`	Customer or worker reported payment happened	`admin_confirmed_paid`, `failed`	`payment_link_sent` unless admin resets manually
`admin_confirmed_paid`	Founder confirmed final payment record	none	Any transition
`failed`	Payment attempt failed or disputed	`payment_link_sent`, `customer_marked_paid`, `waived`	`admin_confirmed_paid` without new report
`waived`	Payment intentionally not collected	none	Any paid state

Complaint state machine

State	Meaning	Allowed transitions	Invalid transitions
`open`	Complaint newly filed	`under_review`, `dismissed`	`resolved`, `closed` directly without admin action
`under_review`	Admin investigating	`resolved`, `dismissed`	`open`
`resolved`	Complaint accepted and closed with action/note	`closed`	`open`, `under_review`
`dismissed`	Complaint rejected or no action needed	`closed`	`open`, `under_review`
`closed`	Terminal complaint state	none	Any transition

Security, testing, and MVP freeze

Security

Domain	Rule
Authentication rules	Workers and admins authenticate through Supabase Auth. Worker UI accepts mobile number + password; backend derives deterministic auth email alias and uses password login. Customer PWA remains anonymous in MVP. Admin login is a hidden `/admin/login` route outside this screen spec, protected by `role=admin`.
Authorization rules	Anonymous customers never query the database directly. Public actions must always go through route handlers using `job_ref + phone + track_code`. Workers can access only their own profile, their own dispatch offers, their assigned jobs, and their own earnings. Admin can access all objects.
Invite code rules	Input is uppercased before validation. Code is checked in both Invite Gate and final request creation. One successful job creation consumes one use. Expired or inactive codes must fail server-side even if client-side state looks valid.
File upload rules	Use private Storage buckets only: `job-media` and `worker-documents`. Server rewrites filenames to UUID-based paths. Never trust client MIME or filename. Restrict file types and size at server and bucket layer. Supabase Storage supports private/public access models and bucket-level restrictions, so bucket policy must mirror server validation. [7]
Activity logging rules	Every mutation creates an `activity_logs` row. Log actor type, actor id when present, action, entity type/id, timestamp, and safe metadata. Log failed worker logins, public lookup failures after threshold, approval decisions, dispatch attempt creation/updates, job status changes, payment changes, complaint changes, invite code changes, and admin settings changes. Do not log raw passwords, full ID numbers, or raw track codes.

Domain	Rule
Backup rules	Before every schema migration, export schema SQL. Weekly export CSV or SQL dump of `worker_profiles`, `jobs`, `payments`, `complaints`, `invite_codes`. Monthly full project dump from local machine or Supabase CLI. Media backups can remain deferred in MVP, but job/worker metadata backups are mandatory before public beta.

Testing

Happy path tests

ID	Scenario	Expected result
HP-1	Customer creates a fixed/quote request with valid invite code	Job created, ref + track code displayed
HP-2	Customer uploads 2 issue photos	Photos stored, linked, visible in admin job details
HP-3	Admin triages job and sends one serial offer	Dispatch attempt created, job enters `dispatching/offer_pending`
HP-4	Worker accepts offer	Job assigned automatically; worker sees Active Job
HP-5	Worker starts work	Job moves to `in_progress`; timeline updates
HP-6	Worker completes work and records collected cash	Job moves to `completed`; payment status becomes `customer_marked_paid` or remains `due` depending payload
HP-7	Admin confirms payment	Payment status `admin_confirmed_paid`; job eligible to close
HP-8	Customer rates worker after closure	One rating row created; worker average updates
HP-9	Admin creates assisted booking from WhatsApp/call	Job created with `request_source=whatsapp_assisted|call_assisted`
HP-10	Worker completes KYC and admin approves	Worker dashboard becomes active

Validation tests

ID	Scenario	Expected result
V-1	Invalid invite code	Request blocked
V-2	Wrong phone format	Field-level error

ID	Scenario	Expected result
V-3	Description less than 20 chars	Field-level error
V-4	Non-serviceable locality selected	Submission blocked
V-5	More than 3 customer photos	Upload blocked
V-6	Worker uploads unsupported file type	Upload blocked
V-7	Rating outside 1–5	Submission blocked
V-8	Complaint description too short	Submission blocked
V-9	Payment amount not equal to final amount	Confirmation blocked
V-10	Duplicate rating on same job	Conflict error

Permission tests

ID	Scenario	Expected result
P-1	Anonymous customer tries to access admin endpoint	401/403
P-2	Worker tries to fetch another worker’s job	403
P-3	Worker tries to approve self	403
P-4	Customer tries lookup with wrong track code	401/404
P-5	Non-admin user hits activity logs endpoint	403
P-6	Suspended worker attempts login	403
P-7	Worker tries to change approved profile fields not allowed by policy	403 or validation failure
P-8	Public client uploads file without valid job credentials	403

Failure scenarios

ID	Scenario	Expected result
F-1	Customer closes browser after request create but before seeing track code	Track flow still works if they have job ref + phone + track code; otherwise founder recovery required for beta
F-2	Photo upload partly fails	API returns partial failure; retry allowed
F-3	Worker accepts expired offer	Conflict / invalid state
F-4	Admin tries to send second active offer before resolving first	Conflict

ID	Scenario	Expected result
F-5	Worker tries to complete job before starting	Invalid transition
F-6	Customer confirms payment before job completed	Invalid transition
F-7	Complaint created after job cancelled before assignment	Invalid transition
F-8	Admin deactivates locality with open jobs	Existing jobs continue; new jobs blocked
F-9	Password reset issued while worker logged in	Force next login with new credentials; old session invalidated if possible
F-10	Network loss during worker status update	No duplicate state changes on retry; endpoint must be idempotent where possible

Beta acceptance checklist

Checklist item	Pass condition
Closed beta gate works	No request can be created without a valid active invite code
Worker onboarding works end-to-end	Admin creates worker → worker logs in → submits KYC → admin approves
Public request flow works on Android Chrome	Form usable on low-end mobile
Manual assisted booking works	Founder can create phone/WhatsApp-origin requests
Serial dispatch works	Exactly one active offer per job enforced in DB and API
Job tracker works without customer account	Lookup works using `job_ref + phone + track_code`
Payment recording works for cash and manual link use case	Both produce correct payment states
Complaint flow works	Complaint appears in admin queue and can be resolved
Activity logs are visible	Critical actions appear with timestamp and actor
Admin can operate entire beta solo	No flow requires external ops employee

MVP freeze

Included in v1.0

Included feature	Scope locked?
Invite-gated customer PWA	Yes
Anonymous customer booking	Yes
Customer issue photo upload	Yes
Customer job tracking using `job_ref + phone + track_code`	Yes
Worker login with mobile number UI + password auth backend	Yes
Worker KYC/profile + document upload	Yes
Founder approval/rejection of workers	Yes
Founder-created assisted bookings	Yes
Serial dispatch, one active offer at a time	Yes
Worker accept/decline offer	Yes
Worker start/complete/report issue	Yes
Payment record with cash / UPI / manual Razorpay link support	Yes
Customer rating	Yes
Customer complaint	Yes
Founder settings for categories/localities/invite codes	Yes
Activity logs	Yes

Explicitly excluded from v1.0

Excluded feature	Reason
Native apps	PWA-first constraint
Customer account login/signup	Not required for closed beta validation
SMS OTP	Cost and provider dependency
WhatsApp API automation	Manual WhatsApp only
IVR/call-center software	Founder handles manually outside product
Public worker directory or choose-your-worker flow	Not the locked operating model
In-app chat	Scope bloat
Maps, pin-drop, routing, live location	Cost and complexity
Live status push notifications	Can use polling/manual contact

Excluded feature	Reason
Referral codes, coupons, ads, subscriptions	Explicitly excluded
Wallet, escrow, payout automation	Explicitly excluded
Public reviews marketplace page	Not needed for beta
Parallel multi-offer bidding	Operational complexity
Automated pricing engine	Founder-led pricing only
AI features	Explicitly excluded

Delayed until 100 jobs

Delayed feature	Why later
Worker availability toggle	Useful only after enough active supply
Customer saved addresses	Only after repeat demand appears
Customer repeat booking shortcut	Premature before usage pattern is clear
Automated email/WhatsApp templates inside admin	Manual copy is enough in early beta
Better queue pagination and exports	Only if founder feels list pain
Worker-to-customer rating	Trust useful, but not needed for first validation
Basic analytics charts	Current dashboard counters are enough

Delayed until 1000 jobs

Delayed feature	Why later
WhatsApp API / automation	Not needed for closed beta
Push notifications	Only valuable at higher offer velocity
Public worker marketplace/discovery	Strategy says no now
Automated dispatch ranking	Requires more data and training feedback
Native apps	Only when PWA becomes limiting
Real-time subscriptions for dashboards	Polling/manual refresh sufficient early
Payment gateway integration beyond manual links	Only if online share rises meaningfully
Settlement and payout automation	Not needed in first controlled operations phase
B2B account workflows	Premature for solo-founder launch
Full multilingual CMS	Hardcoded copy dictionary is enough initially

This PRD is now implementation-ready for database schema generation, API contract generation, UI design, and sprint breakdown.

---

1 Next.js Docs: App Router
https://nextjs.org/docs/app?utm_source=chatgpt.com

2 4 Password-based Auth | Supabase Docs
https://supabase.com/docs/guides/auth/passwords?utm_source=chatgpt.com

3 Metadata Files: manifest.json
https://nextjs.org/docs/app/api-reference/file-conventions/metadata/manifest?utm_source=chatgpt.com

5 7 Storage Buckets | Supabase Docs
https://supabase.com/docs/guides/storage/buckets/fundamentals?utm_source=chatgpt.com

6 Row Level Security | Supabase Docs
https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com